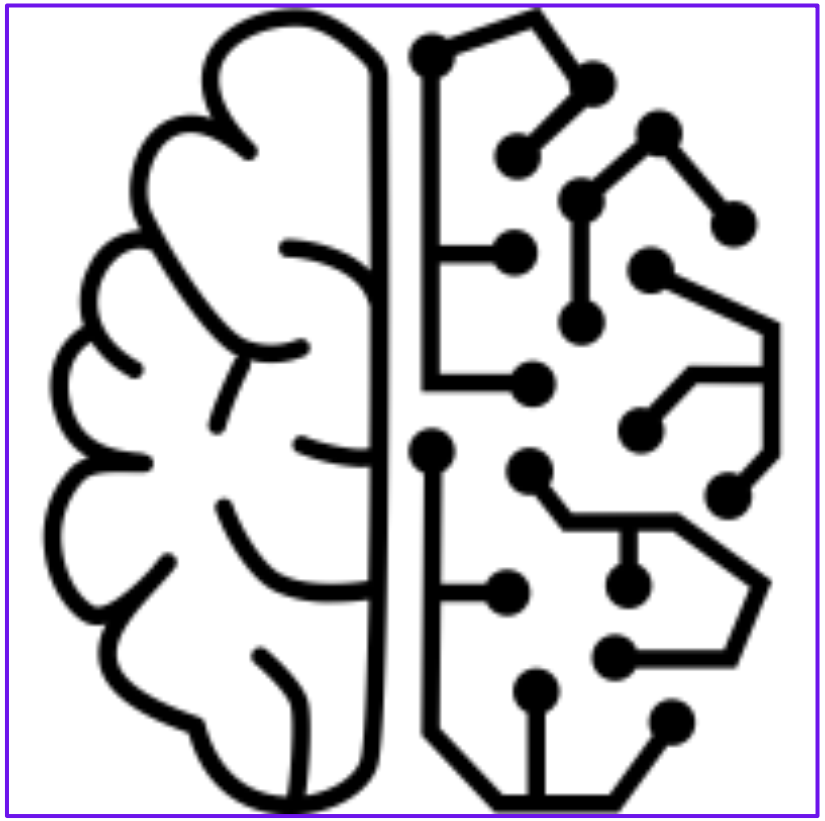

Application web pour visualiser les données

M2

NEXA Digital School

Compétences visées

- Git et versioning du code
- Préparation au déploiement
- Déploiement sur Streamlit Cloud
- Sécurité et bonnes pratiques
- Tests utilisateurs et conclusion

Ressources

NEXA Digital School

Programme du Jour

- Matin (9h - 12h30)
  - Git et versioning du code
  - Préparation au déploiement
- Après-midi (13h30 - 17h00)
  - Déploiement sur Streamlit Cloud
  - Sécurité et bonnes pratiques
  - Tests utilisateurs et conclusion
- **Objectif** : Mettre l'application en production, accessible via un simple lien.

GIT ET VERSIONING

NEXA Digital School

Pourquoi Git est essentiel ?

- **Les problèmes sans Git**
  - `app_v1.py`, `app_v2_final.py`, `app_v2_final_VRAIMENT.py`
  - Impossible de revenir en arrière proprement
  - Travail en équipe chaotique
  - Pas de trace des modifications
  - Risque de perte de code
- Les solutions avec Git
  - Historique complet des modifications
  - Retour en arrière facile
  - Collaboration structurée
  - Branches pour tester sans casser
  - Backup automatique sur GitHub

En entreprise

Git est obligatoire dans 99% des entreprises tech. C'est une compétence attendue.

Les concepts clés de Git

Repository (repo)
Un dossier qui contient votre projet + l'historique Git

Commit
Un "snapshot" de votre code à un instant T
Comme une sauvegarde avec un message explicatif

Branch
Une ligne de développement parallèle
Par défaut : `main` (ou `master`)

Remote
Un dépôt distant (ex: sur GitHub)
Permet de synchroniser avec le cloud

Workflow classique

- Modifier le code localement
- `git add` : Préparer les fichiers à commiter
- `git commit` : Sauvegarder les modifications
- `git push` : Envoyer vers GitHub

Installation et configuration Git

Vérifier si Git est installé
Si pas installé :

- **Windows** : https://git-scm.com/download/win
- **Mac** : brew install git ou Xcode Command Line Tools
- **Linux** : `sudo apt install git`

Configuration initiale (une seule fois)

```
# Configurer votre identité
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@example.com"

# Vérifier la configuration
git config --list
```

Éditeur par défaut (optionnel)

```
git config --global core.editor "code --wait"  # VSCode
```

Initialiser un repository Git

- **Naviguer vers votre projet**
  - `cd /chemin/vers/votre/projet`
- **Initialiser Git**
  - `git init`

    Cette commande crée un dossier caché `.git` qui contient tout l'historique.

- **Vérifier le statut**
  - `git status`

Vous devriez voir tous vos fichiers en rouge (non trackés).

Le fichier .gitignore

Le fichier .gitignore

Pourquoi ?
Certains fichiers ne doivent **jamais** être versionnés :

- Données sensibles (`.env`, mots de passe)
- Fichiers générés (`__pycache__`, `.pyc`)
- Fichiers temporaires (`.DS_Store`, `Thumbs.db`)
- Gros fichiers (datasets, modèles si > 100 MB)

Créer un .gitignore

Créer un fichier `.gitignore` à la racine du projet :

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
*.so
*.egg
*.egg-info/
dist/
build/

# Environnement
.env
.venv
env/
venv/

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db

# Streamlit
.streamlit/secrets.toml

# Data & Models (optionnel si gros fichiers)
# *.csv
# *.pkl

# Logs
*.log
```

Premier commit

1. Ajouter les fichiers

```
# Ajouter tous les fichiers (sauf ceux dans .gitignore)
git add .

# Ou ajouter des fichiers spécifiques
git add app.py
git add requirements.tx
```

2. Vérifier ce qui va être commité

```
git status
```

Les fichiers doivent être en vert maintenant (staged).

3. Créer le commit

```
git commit -m "Initial commit - Application Streamlit fonctionnelle"
```

4. Vérifier l'historique

```
git log
```

NEXA Digital School

Se connecter à GitHub

Créer un compte GitHub

- Aller sur https://github.com
- Sign up (gratuit)
- Confirmer l'email

Créer un Personal Access Token (PAT)

Depuis 2021, GitHub n'accepte plus les mots de passe pour Git.

Étapes :

- GitHub → Settings (en haut à droite)
- Developer settings (tout en bas)
- Personal access tokens → Tokens (classic)
- Generate new token
- Sélectionner les scopes : `repo`, `workflow`
- Generate token
- **COPIER LE TOKEN** (il ne sera plus jamais affiché !)

Créer un repository sur GitHub

Créer un repository sur GitHub

- **Via l'interface web**
  - GitHub → New repository (bouton vert)
  - Nom du repo : `loan-approval-app` (ou autre)
  - Description : "ML app for loan approval prediction"
  - Public ou Private : au choix
  - ❌ NE PAS cocher "Initialize with README" (on a déjà un repo local)
  - Create repository

Lier le repo local au repo GitHub

```
# Ajouter le remote (remplacer USERNAME et REPO)
git remote add origin https://github.com/USERNAME/REPO.git

# Vérifier
git remote -v

# Pousser le code (première fois)
git push -u origin main
```

Si demandé, entrer :

- Username : votre username GitHub
- Password : **votre token** (PAS votre mot de passe)

NEXA Digital School

Workflow Git quotidien

Faire des modifications

```
# 1. Modifier votre code (ex: ajouter une feature)

# 2. Voir ce qui a changé
git status
git diff  # Voir les changements ligne par ligne

# 3. Ajouter les modifications
git add .

# 4. Commiter
git commit -m "Ajout validation des inputs dans le formulaire"

# 5. Pousser vers GitHub
git push
```

Récupérer les modifications (si travail en équipe)

```
git pull
```

Voir L'historique

```
git log -online        # Format compact
git log -graph         # Format graphique
```

NEXA Digital School

TP : Versionner votre projet avec Git

Objectifs

- Initialiser un repo Git dans votre projet
- Créer un fichier `.gitignore` adapté
- Faire votre premier commit
- Créer un repo GitHub
- Pousser votre code sur GitHub
- Vérifier que tout est en ligne

Livrables

- Repository GitHub avec votre code
- Au moins 1 commit avec un message clair
- Fichier `.gitignore` correct

Aide

- Template `.gitignore` fourni
- Assistant disponible pour les problèmes

PRÉPARATION AU DÉPLOIEMENT

Streamlit Cloud : Présentation

Qu'est-ce que c'est ?

Plateforme gratuite d'hébergement pour applications Streamlit
Développée par Streamlit

- Avantages
  - **Gratuit** : Plan Community sans carte bancaire
  - **Simple** : Connexion directe avec GitHub
  - **Rapide** : Déploiement en 1 clic
  - **Automatique** : Redéploiement à chaque push Git
  - **HTTPS** : Sécurisé par défaut
  - **URL propre** : votre-app.streamlit.app

- Limites (plan gratuit)
  - 1 app publique gratuite
  - 3 apps privées gratuites
  - CPU/RAM limités (suffisant pour ce projet)
  - 1 Go de stockage

Fichiers nécessaires au déploiement

1. **requirements.txt (obligatoire)**

- Taille raisonnable pour l'apprentissage
- Problématique réelle et compréhensible
- Permet d'aborder les biais (genre, statut marital)

```
streamlit==1.30.0
pandas==2.1.4
numpy==1.26.3
plotly==5.18.0
scikit-learn==1.3.2
joblib==1.3.2
```

Génération automatique :

`pip freeze > requirements.txt`

⚠️ **Problème :** Inclut TOUTES les librairies, même inutiles

Solution : Créer manuellement (recommandé)

2. packages.txt (optionnel)

Pour installer des packages système (apt-get)

Exemple:

`libgomp1`

NEXA Digital School

Fichiers nécessaires (suite)

1. .streamlit/config.toml (optionnel)

Configuration de l'app

```toml
[theme]
primaryColor = "#FF4B4B"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
textColor = "#262730"
font = "sans serif"

[server]
headless = true
port = 8501
enableXsrfProtection = true
enableCORS = false
```

4. README.md (recommandé)

Documentation du projet

```markdown
# Loan Approval Prediction App

Application web de prédiction d'approbation de prêt utilisant le Machine Learning.

## Fonctionnalités
- Exploration de données interactive
- Prédiction en temps réel
- Visualisations interactives avec Plotly
- Conforme RGPD et Ethics by Design

## Technologies
- Python 3.10
- Streamlit
- Scikit-learn
- Plotly

## Installation locale
...
```

NEXA Digital School

Structure de projet recommandée

Bonnes pratiques

- Code dans `app.py` (ou modules séparés)
- Données dans `data/`
- Modèles dans `models/`
- Pas de chemins absolus (`/Users/monnom/...`)
- Chemins relatifs (`./data/loan_data.csv`)

```
loan-approval-app/
│
├── app.py                    # Application principale
├── requirements.txt          # Dépendances
├── README.md                 # Documentation
├── .gitignore                # Fichiers à ignorer
│
├── .streamlit/
│   ├── config.toml           # Configuration Streamlit
│   └── secrets.toml          # Secrets (LOCAL UNIQUEMENT)
│
├── data/
│   └── loan_data_clean.csv   # Dataset (si < 100 MB)
│
├── models/
│   ├── model_lr.pkl          # Modèle Logistic Regression
│   ├── model_rf.pkl          # Modèle Random Forest
│   └── scaler.pkl            # Scaler
│
├── utils/
│   ├── __init__.py
│   ├── data_processing.py    # Fonctions de preprocessing
│   └── model_utils.py        # Fonctions liées au modèle
│
└── assets/
    ├── logo.png
    └── screenshots/
```

Gestion des secrets et configuration

- Le problème des secrets
  ❌ NE JAMAIS commiter :
  - Clés API (OpenAI, AWS, etc.)
  - Mots de passe de base de données
  - Tokens d'authentification
  - Informations sensibles

La solution : st.secrets

1. Localement : Créer `.streamlit/secrets.toml`

```
# .streamlit/secrets.toml (LOCAL, dans .gitignore)
[api_keys]
openai = "sk-..."
google_maps = "AIza..."

[database]
host = "localhost"
user = "admin"
password = "motdepasse"
```

2. Dans le code : Lire avec `st.secrets`

```
import streamlit as st

# Accéder aux secrets
api_key = st.secrets["api_keys"]["openai"]
db_password = st.secrets["database"]["password"]
```

3. Sur Streamlit Cloud : Ajouter via l'interface web

Optimisation pour le déploiement

1. Réduire la taille des fichiers

Modèles : Compresser si possible

```python
# Au lieu de sauvegarder le modèle entier
joblib.dump(model, 'model.pkl', compress=3)  # Compression niveau 3
```

Dataset : Ne pas inclure si > 100 MB

```python
# Option 1 : Dataset sur GitHub (si < 100 MB)
df = pd.read_csv("data/loan_data.csv")

# Option 2 : Dataset externe (Google Drive, S3, etc.)
@st.cache_data
def load_data():
    url = "https://drive.google.com/uc?id=YOUR_FILE_ID"
    return pd.read_csv(url)
```

2. Optimiser le cache

```python
@st.cache_data(ttl=3600)  # Cache pour 1 heure
def load_large_data():
    return pd.read_csv("big_data.csv")

@st.cache_resource
def load_model():
    return joblib.load("model.pkl")
```

Tester localement avant de déployer

Checklist pré-déploiement

- **L'app fonctionne localement**
  `streamlit run app.py`
  - Aucune erreur
  - Toutes les fonctionnalités marchent
  - Pas de chemins absolus
- **Dépendances correctes**

```
# Créer un environnement virtuel frais
python -m venv test_env
source test_env/bin/activate  # Linux/Mac
test_env\Scripts\activate     # Windows

# Installer depuis requirements.txt
pip install -r requirements.txt

# Tester l'app
streamlit run app.py
```

- **.gitignore correct** secrets.toml dans .gitignore
  Pas de données sensibles commitées
- **README à jour** Explique le projet
  Instructions d'installation

NEXA Digital School

TP : Préparer le déploiement

- Objectifs
  - Créer un fichier `requirements.txt` correct
  - Créer un `README.md` complet
  - Créer `.streamlit/config.toml` (optionnel)
  - Vérifier que `.gitignore` contient `secrets.toml`
  - Vérifier la structure du projet
  - Tester l'app dans un environnement frais
  - Commiter et pousser sur GitHub

- Livrable
  - `requirements.txt` avec dépendances exactes
  - `README.md` documenté
  - Structure de projet propre
  - Code sur GitHub, prêt à déployer

- Template fourni
  - Template `requirements.txt`
  - Template `README.md`
  - Template `config.toml`

DÉPLOIEMENT SUR STREAMLIT CLOUD

Créer un compte Streamlit Cloud

Étapes

- Aller sur https://share.streamlit.io/
- Cliquer sur "Sign up"
- Utiliser GitHub pour se connecter (recommandé)
- Autoriser Streamlit à accéder à vos repos GitHub
- Vous êtes redirigé vers votre dashboard

Dashboard Streamlit Cloud

Vous voyez :

- Vos apps déployées (vide pour l'instant)
- Bouton "New app" pour déployer
- Workspaces (si plusieurs comptes GitHub)

Déployer votre première app

Étape 1 : Cliquer sur "New app"
Vous voyez un formulaire avec 3 champs :

- Repository
  - Sélectionner votre repo GitHub
  - Ex: votre-username/loan-approval-app
- Branch
  - Généralement main (ou master)
- Main file path
  - Chemin vers votre fichier principal
  - Ex: app.py (si à la racine)
  - Ex: src/app.py (si dans un sous-dossier)

Étape 2 : Advanced settings (optionnel)
Python version : 3.10 (ou celle que vous utilisez)
Secrets : On configurera après (voir slide suivant)

Étape 3 : Deploy !
Cliquer sur le bouton "Deploy"

Le processus de déploiement

Ce qui se passe en coulisses

1. Clone du repo GitHub
2. Détection de Python
3. Installation des dépendances
4. Lancement de l'app
5. Attribution d'une URL

Durée
Généralement 2-5 minutes pour le premier déploiement.

Votre app est en ligne !

Tester l'app

- Cliquer sur l'URL générée : https://votre-app.streamlit.app
- L'app s'ouvre dans le navigateur
- Tester toutes les fonctionnalités :
  - Navigation entre onglets
  - Visualisations
  - Formulaire de prédiction
  - Tout doit fonctionner comme en local

Partager l'app

Copier l'URL et partager :

- Par email
- Sur LinkedIn
- Dans votre portfolio
- CV (section Projets)

URL personnalisée (optionnel)

Possible de personnaliser : `https://loan-predictor.streamlit.app`

Settings → General → App URL

Configurer les secrets sur Streamlit Cloud

Si votre app utilise des secrets

Scénario : Vous avez des API keys, mots de passe, etc.

Étapes

- Dashboard Streamlit Cloud → Votre app
- Settings (⚙️) → Secrets
- Coller le contenu de votre fichier `secrets.toml` local
- Save
- L'app redémarre automatiquement avec les secrets

Sécurité

- Les secrets sont chiffrés
- Jamais exposés dans les logs
- Jamais visibles publiquement
- Accessibles uniquement via `st.secrets` dans le code

Redéploiement automatique

Git push → Redéploiement automatique

Workflow :

- Modifier le code localement
- Tester en local : `streamlit run app.py`
- Commiter : `git commit -m "Ajout nouvelle feature"`
- Pousser : `git push`
- Streamlit Cloud détecte le push
- **Redéploiement automatique** en 1-2 minutes

Voir les déploiements
Dashboard → Votre app → "Manage app" → "Reboot"

Logs en temps réel
Cliquer sur "Manage app" pour voir les logs de déploiement

Forcer un redéploiement
Si un push ne déclenche pas le redéploiement :

Settings → "Reboot app"

TP : Déployer votre application

Objectifs

- Créer un compte Streamlit Cloud (si pas déjà fait)
- Déployer votre app en suivant les étapes
- Tester l'app en ligne
- Corriger les éventuelles erreurs
- Partager l'URL avec vos camarades
- (Bonus) Personnaliser l'URL de l'app

Critères de réussite

- App accessible via une URL publique
- Toutes les fonctionnalités marchent
- Pas d'erreur dans les logs
- URL partagée avec au moins 2 personnes

Aide disponible

- Guide étape par étape projeté
- Assistant pour déboguer les erreurs

SÉCURITÉ ET BONNES PRATIQUES

Sécurité : Rappels HTTPS

C'est quoi HTTPS ?

HTTP : Données en clair
HTTPS : Données chiffrées (SSL/TLS)

Streamlit Cloud = HTTPS par défaut

- Toutes les apps Streamlit Cloud sont en HTTPS
- Certificat SSL automatique
- Pas de configuration nécessaire
- Les données entre l'utilisateur et le serveur sont chiffrées

Vérifier
Regarder la barre d'adresse : 🔒 https://votre-app.streamlit.app

- Pourquoi c'est important ?
  - Protection contre l'interception (man-in-the-middle)
  - Confiance des utilisateurs (cadenas vert)
  - SEO (Google favorise HTTPS)
  - Conformité RGPD

Sécurité : Validation des inputs

Le problème

Les utilisateurs peuvent entrer n'importe quoi :

- Nombres négatifs
- Valeurs absurdes (revenu à 1 milliard)
- Types incorrects
- Tentatives d'injection

Solutions implémentées

1. Contraintes sur les widgets

```
st.number_input("Revenu", min_value=0, max_value=1000000)
```

2. Validation manuelle

```
if income < 1000:
    st.warning("Le revenu semble très faible")

if loan_amount > income * 100:
    st.error("Montant du prêt trop élevé par rapport au revenu")
```

3. Sanitization

```
# Éviter les injections SQL (si DB)
import re
def sanitize_input(text):
    return re.sub(r'[^\w\s-]', '', text)
```

NEXA Digital School

Ce qui est loggé par défaut

Streamlit Cloud logge automatiquement :

- Démarrages de l'app
- Erreurs Python
- Warnings
- Print statements

Ce qui NE DOIT PAS être loggé

- Données personnelles (noms, emails, etc.)
- Secrets (API keys, passwords)
- Informations sensibles (montants exacts, scores)

Bonnes pratiques

MAUVAIS

```
print(f"Prédiction pour {user_name} : {prediction}")
st.write(f"API Key: {api_key}")
```

BON

```
print(f"Prédiction effectuée : {prediction}")
st.write("Connexion API réussie")
```

Logs de production

Pour une app en production, utiliser un système de logging professionnel :

- `logging` module Python
- Rotation des logs
- Anonymisation des données

Sécurité : CORS et accès externe

C'est quoi CORS ?

Cross-Origin Resource Sharing : Mécanisme de sécurité des navigateurs
Par défaut, un site A ne peut pas faire de requêtes AJAX vers un site B.

Streamlit et CORS
Streamlit Cloud gère CORS automatiquement pour :

- Les requêtes depuis l'app elle-même
- Les APIs publiques

Configuration CORS (si nécessaire)
Dans `.streamlit/config.toml`:

```
[server]
enableCORS = false
enableXsrfProtection = true
```

Best practices

- Utiliser enableXsrfProtection = true (protection CSRF)
- Ne pas désactiver CORS sans raison
- Si API custom, configurer CORS sur l'API

NEXA Digital School

Sécurité : Gestion des rôles et

Apps publiques

- Accessibles par TOUS via l'URL
- Pas d'authentification
- OK pour des démos, portfolios

Apps privées

- Accessibles uniquement par les personnes autorisées
- Authentification via GitHub
- Nécessite un repo GitHub privé

Ajouter de l'authentification custom

Option 1 : Simple mot de passe

```python
import streamlit as st

password = st.text_input("Mot de passe", type="password")

if password == st.secrets["app_password"]:
    # Afficher l'app
    st.success("Accès autorisé")
    # ... reste de l'app
else:
    st.error("Accès refusé")
    st.stop()
```

Option 2 : OAuth (Google, GitHub, etc.)
Utiliser une librairie comme streamlit-authenticator

Option 3 : Intégration SSO entreprise
Pour des apps corporate, intégrer avec l'SSO de l'entreprise

Bonnes pratiques : Performance

1. Utiliser le cache intelligemment

```python
@st.cache_data(ttl=3600)  # Cache pour 1 heure
def load_data():
    return pd.read_csv("data.csv")

@st.cache_resource
def load_model():
    return joblib.load("model.pkl")
```

2. Limiter les calculs inutiles

MAUVAIS : Recalculer à chaque interaction

```python
df = expensive_computation()
st.write(df)
```

```python
if 'df' not in st.session_state:
    st.session_state.df = expensive_computation()
st.write(st.session_state.df)
```

3. Lazy loading

Charger les données seulement quand nécessaire :

```python
if st.button("Voir les détails"):
    details = load_detailed_data()  # Charge seulement si bouton cliqué
    st.write(details)
```

NEXA Digital School

Bonnes pratiques : Performance (suite)

4. Optimiser les visualisations

Pour de gros datasets :

```
# Limiter le nombre de points affichés
df_sample = df.sample(1000)  # Afficher 1000 points au lieu de 100k
fig = px.scatter(df_sample, x='x', y='y')
```

Choisir le bon graphique :

- Plotly pour l'interactivité
- Matplotlib pour des graphiques statiques simples (plus léger)

5. Pagination

Pour de grands datasets :

```
page_size = 100
page = st.number_input("Page", min_value=1,
max_value=len(df)//page_size)
start_idx = (page - 1) * page_size
end_idx = start_idx + page_size
st.dataframe(df.iloc[start_idx:end_idx])
```

6. Monitoring

Utiliser st.experimental_get_query_params() pour tracker l'usage :

```
params = st.experimental_get_query_params()
if 'source' in params:
    # Log d'où vient l'utilisateur
    pass
```

Bonnes pratiques : Monitoring et analytics

Métriques à suivre en production

1. Utilisation

- Nombre de visiteurs
- Pages vues
- Durée de session

2. Performance

- Temps de chargement
- Taux d'erreur
- Utilisation CPU/RAM

3. Business

- Nombre de prédictions effectuées
- Taux de conversion (si applicable)
- Features les plus utilisées

Outils

Google Analytics (gratuit)

```
# Ajouter dans app.py
st.components.v1.html("""
<script async
src="https://www.googletagmanager.com/gtag/js?
id=GA_MEASUREMENT_ID"></script>
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', 'GA_MEASUREMENT_ID');
</script>
""", height=0)
```

Streamlit Cloud Analytics (intégré) Dashboard → App → Analytics

README.md complet

Doit contenir :

- **Description** : Que fait l'app ?
- **Fonctionnalités** : Liste des features
- **Technologies** : Stack technique
- **Installation locale** : Comment l'exécuter en local ?
- **Déploiement** : Comment la déployer ?
- **Usage** : Comment l'utiliser ?
- **Screenshots** : Captures d'écran
- **Auteur** : Qui l'a créé ?
- **License** : MIT, Apache, etc.

Documentation du code

```python
def predict_loan_approval(features):
    """
    Prédit l'approbation d'un prêt.

    Args:
        features (pd.DataFrame): Features du demandeur

    Returns:
        tuple: (prediction, probability)

    Example:
        >>> features = pd.DataFrame({...})
        >>> pred, proba = predict_loan_approval(features)
        >>> print(pred)  # 1 (Approved)
    """
    # Code...
```

TP 4 :

Objectifs

- Vérifier que votre app n'a pas de failles de sécurité :
  - Pas de secrets dans le code
  - Validation des inputs
  - Pas de données sensibles dans les logs
- Optimiser la performance
  - Vérifier que cache est bien utilisé
  - Tester le temps de chargement
  - Optimiser si nécessaire
- Améliorer la documentation :
  - Compléter le README avec screenshots
  - Ajouter docstrings aux fonctions principales
  - Ajouter une section "Démo" avec l'URL de l'app
- Tester l'app en conditions réelles :
  - Ouvrir sur mobile
  - Tester sur différents navigateurs
  - Demander à 2 personnes de tester